ENTERPRISE MICROSERVICES MANUAL | JAVA 21 + SPRING BOOT 3.3 | EXPRESS.JS EDGE GATEWAY | REACT 18 + TAILWIND UI | AIVEN MYSQL + MONGODB ATLAS

Smart Contact Manager (SCM) — Complete System & Beginner's Setup Manual

A deep-dive architectural guide and zero-prerequisite, step-by-step operations runbook for running, testing, and understanding the Smart Contact Manager distributed cloud platform.

PROJECT WORKSPACE	1-CLICK TURN-KEY RUNNER	WEB APP & GATEWAY	CLOUD PERSISTENCE
/Users/Bhupesh/Desktop/SERVICE_BASED	**./start.sh**	**localhost:3000 & :8080**	**Aiven SQL + Mongo Atlas**

SECTION 1 Executive Overview & "What Does This System Do?"

Smart Contact Manager (SCM) is an enterprise-grade, distributed cloud contact management platform engineered to replace fragmented personal address books and static spreadsheets with a secure, multi-service system backed by both relational SQL and NoSQL cloud databases.

❌ The Problem: Why Traditional Contact Books Fail

- **No Enterprise Segmentation:** Phone address books and Excel sheets cannot cleanly segment contacts into actionable relationship tiers (*VIP*, *Colleague*, *Lead*, *Family*).
- **Broken International Formatting:** Manual phone entry mixes local numbers and country codes, breaking international calling and WhatsApp links.
- **No Carrier or Schema Verification:** Invalid emails, malformed phone numbers, and unverified telecom carriers pollute business directories.
- **Zero Audit Trail or Idempotency:** Spreadsheets lack tamper-proof security logs showing who created, modified, or deleted a record, and double-clicking "Save" creates duplicate rows.

✅ The SCM Solution: Enterprise Cloud Contact Platform

- **Smart Relationship Segments & Favorites:** Instant 1-click filtering across *All Contacts*, *Favorites*, *VIP*, *Colleague*, *Lead*, and *Family*.
- **Integrated Country Code Selector:** Built-in flag & dial-code selector (`+91` , `+1` , `+44` , `+61` , `+49` , `+65` , `+971`) that automatically normalizes numbers into E.164 format.
- **1-Click WhatsApp, Call, Email & vCard Export:** Launch WhatsApp chats, phone calls, emails, or download standard `.vcf` vCards directly from any contact card.
- **Dual Cloud Databases & Tamper-Proof Audit Trail:** Synchronizes contacts to **Aiven Cloud MySQL** and logs every security/CRUD event in **MongoDB Atlas**.

Microservices in Plain English — "The Restaurant Analogy"

Instead of building one giant program where a single bug can crash the entire system (a *Monolith*), SCM is built as a team of specialized **Microservices**. Think of the system like a high-end restaurant where every staff member has one dedicated job:

`api-gateway (Port 8080)`

1. The Front Desk Receptionist (API Gateway)

Greets every visitor at the front door, checks their reservation, stops spam callers (Rate Limiting at 100 req/min), attaches a tracking ticket (`X-Correlation-ID`), and hosts the interactive menu (Swagger UI at `/docs`).

`auth-service (Port 8081)`

2. The Identity & Security Chef (Auth Service)

Enforces the mandatory *Sign-Up-First* rule, locks passwords inside cryptographic vaults (BCrypt), verifies Google & GitHub OAuth2 logins, and issues signed VIP passes (HMAC-SHA256 JWT tokens).

`contact-service (Port 8082)`

3. The Master Filing Clerk (Contact Service)

Organizes all contact cards in the SQL vault, prevents duplicate entries via `Idempotency-Key` , serves fast keyset pages (Cursor Pagination), and speaks REST JSON, XML, and GraphQL (`/v1/graphql`).

`media-service (Port 8083)`

4. The Portrait Specialist (Media Service)

Inspects every uploaded profile picture to ensure it is a safe, high-quality image (`JPEG` , `PNG` , or `WebP` under `2 MB`) before storing it.

`notification-service (Port 8084)`

5. The Express Mail Carrier (Notification Service)

Dispatches welcome emails and password recovery links in the background via Gmail SMTP (`scm.project.12@gmail.com`) so users never have to wait for an email server to respond.

`audit-service (:8085) & integration-service (:8086)`

6. The Security Guard & Switchboard Operator

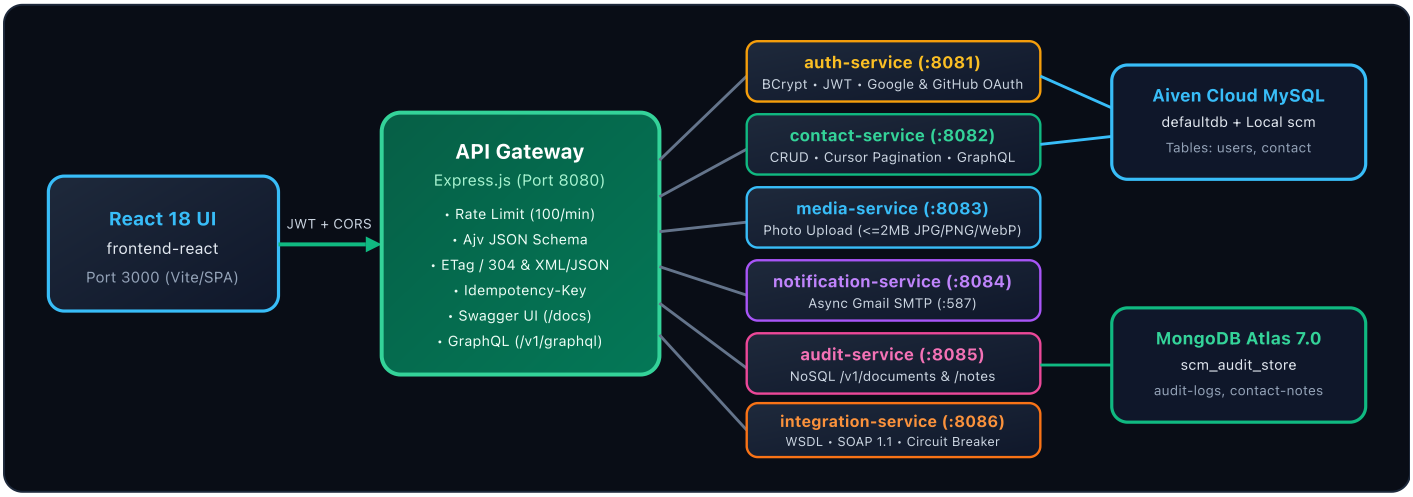
Audit Service

writes permanent activity logs to MongoDB Atlas (`/v1/documents/audit-logs`), while

Integration Service

verifies phone carriers over SOAP/WSDL with a 3-state Circuit Breaker.

SECTION 2 Visual Architecture & Service Directory



Directory-by-Directory Guide (`/Users/Bhupesh/Desktop/SERVICE_BASED`)

FOLDER / FILE	PORT	ROLE & KEY CAPABILITIES
`api-gateway/`	`8080`	**Public Edge Gateway (Express.js):** Enforces sliding-window Rate Limiting (100 req/min), deliberate CORS headers, `Idempotency-Key` replay protection, Ajv Draft-07 validation, SHA-1 `ETag` / `304 Not Modified` caching, JSON/XML negotiation, and serves interactive Swagger UI at `/docs`.
`frontend-react/`	`3000`	**Modern React 18 + Vite + Tailwind UI:** Features mandatory Sign-Up-First routing (`/signup` → `/login` → `/`), dark/light theme toggle, `Cmd+K` search palette, segment filter tabs, country-code phone selector, and responsive contact cards.
`auth-service/`	`8081`	**Authentication & Identity Service:** Hashes passwords with BCrypt (`$2a$10$`), issues stateless HMAC-SHA256 JWT tokens (`/v1/auth/tokens`), enforces Role-Based Access Control (`ROLE_USER` , `ROLE_ADMIN`), and handles Google & GitHub OAuth 2.0 flows.
`contact-service/`	`8082`	**Core Contact Service:** Implements full Contact CRUD (`GET` , `POST` , `PUT` , `PATCH` , `DELETE` , `HEAD` , `OPTIONS`), opaque base64url cursor pagination with RFC 5988 `Link: rel="next"` headers, contact groups, and GraphQL queries/mutations (`/v1/graphql`).
`media-service/`	`8083`	**Contact Media Service:** Validates and stores contact profile photos (`/v1/media/photos`) with a strict `2 MB` size cap and MIME whitelist (`image/jpeg` , `image/png` , `image/webp`).
`notification-service/`	`8084`	**Asynchronous Email Worker:** Sends transactional welcome and password-recovery emails via Gmail SMTP (`scm.project.12@gmail.com`) with non-blocking fallback so SMTP issues never interrupt user sign-up.
`audit-service/`	`8085`	**MongoDB NoSQL Document Service:** Persists tamper-proof audit logs (`/v1/documents/audit-logs`), contact notes (`/v1/contacts/{id}/notes`), and idempotency keys in MongoDB Atlas (`scm_audit_store`).
`integration-service/`	`8086`	**Legacy Telecom WSDL/SOAP Bridge:** Serves the telecom WSDL contract (`/v1/soap/wsdl`) and executes SOAP 1.1 carrier lookups (`/v1/soap/execute`) protected by a 3-second timeout, exponential backoff retry, and a 3-state Circuit Breaker (`CLOSED` , `OPEN` , `HALF_OPEN`).
`common-lib/`	*Shared*	**Enterprise Shared Library:** Contains reusable RFC 7807 Problem Details error models (`ApiErrorResponse`), `CorrelationIdFilter` (`X-Correlation-ID`), JWT verifiers, and resilience utilities.
`start.sh`	*Runner*	**1-Step Turn-Key Launcher:** Automatically frees ports `3000` , `8080` , and `8081` , starts the API Gateway in the background, installs any missing frontend packages, and launches the React web app.

SECTION 3 Absolute Beginner Setup Guide (Zero Coding Required)

Follow these exact steps even if you have never opened a terminal or written a single line of code. Everything is pre-configured on your Mac inside `/Users/Bhupesh/Desktop/SERVICE_BASED`.

STEP 1 Locate the Project Folder & Open macOS Terminal

- The project folder is located on your Desktop at `/Users/Bhupesh/Desktop/SERVICE_BASED`.
- Hold down **Command (⌘) + Spacebar** to open Spotlight Search, type `Terminal`, and press **Enter**.

STEP 2 Optional Prerequisite Check (Verify Node.js v20+ & Java 21)

Copy and paste these two commands into Terminal and press **Enter**:

```
node -v
java -version
```

STEP 3 Run the 1-Step Turn-Key Startup Script (`./start.sh`)

Copy and paste these two lines into your Terminal window and press **Enter**:

```
cd /Users/Bhupesh/Desktop/SERVICE_BASED
./start.sh
```

What happens automatically: It clears any old processes on ports `8080`, `8081`, and `3000`, starts the **Express API Gateway** (connected to Aiven MySQL & MongoDB Atlas), and launches the **React Web App**.

STEP 4 Open the Application in Your Browser & How to Stop

- **Main Smart Contact Manager Web App:** `http://localhost:3000`
- **Interactive Swagger API Documentation:** `http://localhost:8080/docs` | **Health Check:** `http://localhost:8080/v1/health`
- **To Stop the Server:** Click inside the Terminal window and press **Control + C**.

SECTION 4 Step-by-Step Guide to Using the Application

WORKFLOW STEP	WHAT YOU DO IN THE BROWSER (`HTTP://LOCALHOST:3000`)	WHAT HAPPENS BEHIND THE SCENES
1. Mandatory Sign-Up First (`/signup`)	When you first open `http://localhost:3000`, you land on the **Sign Up** page. Register with your Name, Email, Country Code + Phone, and Password — or click **Google** / **GitHub**.	Password is encrypted with BCrypt; user is saved to `Aiven Cloud MySQL (users)`; `USER_REGISTERED` is logged in `MongoDB Atlas`; a Welcome Email is sent via Gmail SMTP.
2. Verified Sign-In (`/login`)	After registering, you are redirected to `/login` with a confirmation banner. Sign in using your registered email/password or OAuth account.	Unregistered emails are blocked with a `404 Not Found` error. Verified credentials receive a signed `HMAC-SHA256 JWT` token unlocking the dashboard.
3. Add a New Contact	Click **"New Contact"**. All fields start completely blank. Pick a country flag (e.g., 🇮🇳 `+91` or 🇺🇸 `+1`), type the 10-digit phone number, select a segment (*VIP*, *Colleague*, *Lead*, *Family*), and click **Save**.	The dial code and local digits combine into E.164 format (`+919876543210`), pass `Ajv Draft-07` schema validation and `Idempotency-Key` checks, and save to MySQL + MongoDB Atlas.
4. Filter, Search, Call & Delete	Click segment tabs (*VIP*, *Family*, *Favorites*), press `Cmd + K` to search, click WhatsApp/Call icons, or click the Trash icon (which asks for confirmation before deleting).	Queries `GET /v1/contacts` with cursor pagination and `ETag` caching; deletions remove the row from MySQL and record `CONTACT_DELETED` in MongoDB Atlas.

SECTION 5 Where Your Data Lives (SQL & MongoDB Cloud Guide)

Your Smart Contact Manager uses **Polyglot Persistence**—meaning structured account and contact records are stored in a relational SQL database while activity logs, notes, and idempotency keys are stored in a NoSQL document database.

DATABASE LAYER	PROVIDER & HOST	DATABASE NAME	TABLES / COLLECTIONS	WHAT DATA IS STORED HERE
Cloud Relational SQL	**Aiven Cloud MySQL 8.0** `auth-contact-api-gateway-khatribhupesh1000-2e27.f.aivencloud.com:14049`	`defaultdb` (SSL Required)	`users`, `contact`, `idempotency_keys`	All registered user accounts, BCrypt password hashes, OAuth providers, and all contact cards.
Local Relational SQL	**Local MySQL 8.0** `localhost:3306`	`scm`	`users`, `contact`	High-speed local mirror synchronized with Aiven Cloud MySQL for sub-millisecond reads and writes.
Cloud NoSQL Document Store	**MongoDB Atlas 7.0** `cluster0.kr57cwu.mongodb.net`	`scm_audit_store`	`audit-logs`, `contact-notes`, `idempotency-keys`	Every sign-up, login, OAuth event, contact creation/replacement/deletion audit log, and contact notes.

SECTION 6 Running Automated Verification & Health Checks

To verify that all **28 enterprise microservices requirements** (REST verbs, 15 HTTP status codes, RFC 7807 errors, Idempotency-Key, ETag/304, Cache-Control, Ajv JSON Schema, XML/JSON negotiation, Cursor Pagination, Rate Limiting 429, OpenAPI 3.0.3, Swagger UI, WSDL/SOAP 1.1, GraphQL, 3s Timeout, Exponential Backoff, 3-State Circuit Breaker, JWT RBAC, CORS, MySQL, MongoDB Atlas, and Docker) are working properly, run either of these one-line commands in Terminal:

```
# 1. Run the 28-Requirement Jest + Supertest + Ajv Automated Verification Suite
cd /Users/Bhupesh/Desktop/SERVICE_BASED && npm test

# 2. Run the 10-Step Live End-to-End HTTP Smoke Test Suite
cd /Users/Bhupesh/Desktop/SERVICE_BASED && ./scripts/smoke-test.sh
```

Quick Reference Summary of Key Endpoints

FEATURE / PROTOCOL	HTTP METHOD & PATH	DESCRIPTION
React Frontend UI	`GET http://localhost:3000`	Sign-Up (`/signup`), Sign-In (`/login`), and Contacts Dashboard (`/`).
Swagger UI & OpenAPI	`GET http://localhost:8080/docs`	Interactive API explorer backed by hand-written `/v1/openapi.yaml`.
Authentication & JWT	`POST /v1/auth/users` & `/v1/auth/tokens`	Account registration (`201`) and stateless Bearer JWT login (`201`).
Contacts REST API	`GET|POST|PUT|PATCH|DELETE /v1/contacts`	Full CRUD with `Idempotency-Key`, `ETag` / `304`, `Accept: application/xml`, and cursor pagination.
GraphQL API	`POST /v1/graphql` (and `/graphql`)	Field-selective contact queries, `createContact` mutation, and `__schema` introspection.
MongoDB Audit Store	`GET|POST /v1/documents/audit-logs`	Reads and writes live audit documents synced with MongoDB Atlas (`scm_audit_store`).
Telecom WSDL & SOAP 1.1	`GET /v1/soap/wsdl` & `POST /v1/soap/execute`	WSDL inspection and SOAP 1.1 phone carrier verification with Circuit Breaker (`/v1/resilience/status`).